

شرح مسئله

در یک شبکه حمل و نقل، حسگرهای نصب شده روی یال‌ها فقط تعداد وسایل نقلیه عبوری را ثبت می‌کنند و مبدأ و مقصد سفرها مشخص نیست. هدف بخش اول پروژه، تخمین ماتریس مبدأ-مقصد از روی شمارش یال‌ها است. داده‌های پروژه به صورت مصنوعی و با یک مقدار اولیه ثابت تولید می‌شوند. ابتدا شبکه، ماتریس مبدأ-مقصد واقعی و شمارش یال‌ها ساخته می‌شوند. ماتریس واقعی فقط برای ارزیابی استفاده می‌شود و الگوریتم تخمین به آن دسترسی ندارد.

در بخش دوم، تقاضای مقصدها از ماتریس تخمینی استخراج می‌شود و مسئله مسیریابی ناوگان با رعایت ظرفیت خودروها حل می‌شود. برای بررسی اثر خطای تخمین، مسیریابی یک بار با تقاضاهای تخمینی و بار دیگر با تقاضاهای واقعی انجام شده و نتایج آن‌ها با یکدیگر مقایسه می‌شوند.

روش ریاضی تخمین ماتریس OD

برای تخمین ماتریس OD از سه داده اصلی استفاده می‌شود:

- بردار OD: شامل تعداد سفرهای هر زوج مبدأ-مقصد
- ماتریس تخصیص P: مشخص کننده یال‌های مورد استفاده در کوتاه ترین مسیر هر سفر
- بردار Link_count شامل شمارش مشاهده شده روی یال‌ها
- درایه های غیر قطری ماتریس OD ابتدا به یک بردار تبدیل می‌شوند:

$$x = \begin{bmatrix} OD_{12} \\ OD_{13} \\ \vdots \\ OD_{n,n-1} \end{bmatrix}$$

هرسطر از ماتریس P مربوط به یک یال و هر ستون آن مربوط به یک زوج مبدأ مقصد است. مقدار هر درایه برابر یک است اگر کوتاه ترین مسیر آن زوج مبدأ مقصد از یال مربوط عبور کند در غیر این صورت مقدار درایه صفر خواهد بود. بنابر این شمارش یال‌ها از ضرب ماتریس تخصیص در بردار OD بدست می‌آید

$$link_count = P \times OD$$

این دستگاه را میتوان به صورت ماتریسی نمایش داد:

$$\begin{bmatrix} link_count_1 \\ link_count_2 \\ \vdots \\ link_count_m \end{bmatrix} = \begin{bmatrix} P_{1,1} & P_{1,2} & \cdots & P_{1,q} \\ P_{2,1} & P_{2,2} & \cdots & P_{2,q} \\ \vdots & \vdots & \ddots & \vdots \\ P_{m,1} & P_{m,2} & \cdots & P_{m,q} \end{bmatrix} \begin{bmatrix} OD_1 \\ OD_2 \\ \vdots \\ OD_q \end{bmatrix}$$

در نتیجه، شمارش بازتولید شده برای هر یال برابر مجموع تقاضاهایی است که مسیر آن‌ها از آن یال عبور می‌کند:

$$\hat{y}_e = \sum_{i \neq j} P_{e,(i,j)} OD_{i,j}$$

از آنجا که تعداد مجهول های OD معمولا بیشتر از تعداد شمارش یال هاست دستگاه زیر معین بوده و ممکن است چند جواب گوناگون داشته باشد. به همین دلیل برای رسیدن به یک جواب پایدار از کمترین مربعات همراه با منظم سازی L2 استفاده میشود:

$$\min_{OD \geq 0} |P_{OD} - link_count|_2^2 + \lambda |OD|_2^2$$

قسمت اول فرمول اختلاف شمارش بازتولید شده و مشاهده شده را کمینه میکند. قسمت دوم جمله منظم سازی است که از تولید مقادیر بسیار بزرگ و ناپایدار جلوگیری میکند قید $OD \geq 0$ نیز تضمین میکند تعداد سفرها منفی نباشند.

این تابع در برنامه با الگوریتم L-BFGS-B حل میشود و نتیجه آن بردار OD-est است. سپس این دو بردار دوباره به فرم ماتریسی تبدیل میشود.

تقاضای گره دپو صفر در نظر گرفته میشود. سپس فاصله میان تمام زوج گره ها با الگوریتم Floyd-Warshall محاسبه میشود بنابراین اگر میان دو گره یال مستقیم وجود نداشته باشد هزینه حرکت میان آنها برابر کوتاه ترین مسیر موجود در شبکه خواهد بود.

حل کننده VRP از یک روش حریصانه استفاده میکند. هر خودرو از دپو شروع میکند و در هر مرحله نزدیکترین مشتری را انتخاب میکند که افزودن تقاضای آن باعث نقض ظرفیت خودرو نشود. پس از آنکه مشتری شدنی دیگری برای خودرو باقی نماند خودرو به دپو باز میگردد و خودروی بعدی شروع به کار میکند. پس از حل مسئله دوش شرط ضروری بررسی میشوند:

$$\sum_j d_j \leq kQ$$

یعنی مجموع تقاضا نباید از ظرفیت کل ناوگان بیشتر باشد. همچنین باید داشته باشیم:

$$\max_j d_j \leq Q$$

یعنی تقاضای هیچ مشتری نباید از ظرفیت یک خودرو بیشتر باشد. روش حریصانه استفاده شده سریع و ساده است و برای داده فعلی جواب معتبر تولید میکند اما لزوما تضمین نمیشود که جواب نهایی از لحاظ هزینه بهینه باشد.

معرفی فایل های پروژه

فایل config.json

این فایل تمام تنظیمات قابل تغییر برنامه را نگهداری میکند مقدار Seed برای تکرار پذیری تولید داده استفاده میشود. بخش network شامل تعداد گره ها احتمال ایجاد یال و محدوده هزینه یال ها است. بخش od_genetator محدوده تولید تعداد سفر و شدت نویز را مشخص میکند بخش estimator مقدار λ را نگهداری میکند و بخش vrp شامل شماره دپو تعداد خودرو ها و ظرفیت خودرو است

فایل generator.py

وظیفه این فایل تولید تمام داده های مصنوعی پروژه است. شبکه جهتدار ماتریس مجاورت ماتریس واقعی OD کوتاه ترین مسیرها ماتریس تخصیص P و شمارش یالها در این فایل تولید میشوند.

فایل estimator.py

این فایل تابع هدف مسئله تخمین را تعریف و بردار OD-est را محاسبه میکند. تخمین زن فقط از ماتریس P شمارش یالها و پارامتر منظم سازی استفاده میکند و به OD_True دسترسی ندارد.

evaluator.py فایل

این فایل کیفیت ماتریس تخمینی را ارزیابی می‌کند. معیارهای MAE، RMSE، خطای نسبی کل و خطای بازتولید شمارش یال‌ها در این فایل محاسبه می‌شوند.

vrp.py فایل

این فایل وظیفه استخراج تقاضا از ماتریس OD، محاسبه ماتریس فاصله، ساخت مسیرهای خودروها، محاسبه هزینه و اعتبارسنجی مسیرها را بر عهده دارد.

main.py فایل

این فایل نقطه اصلی اجرای برنامه است. تمام مراحل تولید داده، تخمین OD، ارزیابی، حل VRP و نمایش نتایج از طریق این فایل اجرا می‌شوند.

requirements.txt فایل

این فایل کتابخانه‌های موردنیاز برنامه، شامل NumPy، SciPy، NetworkX و Colorama را مشخص می‌کند.

توضیح توابع موجود در فایلها

Generator.py

تابع load_config

این تابع مسیر فایل تنظیمات را دریافت کرده و محتوای config.json را میخواند. خروجی تابع یک دیکشنری شامل تمام تنظیمات پروژه است.

تابع Generate_network

این تابع تنظیمات شبکه را دریافت و یک گراف جهت‌دار و وزن‌دار ایجاد می‌کند. ابتدا تمام گره‌ها به گراف اضافه می‌شوند. سپس برای تضمین دسترسی میان گره‌ها، یک چرخه جهت‌دار ساخته می‌شود. در ادامه یال‌های اضافی با احتمال edge_probability ایجاد شده و برای هر یال یک هزینه تصادفی در محدوده تعیین‌شده در نظر گرفته می‌شود. استفاده از random.Random با seed ثابت باعث می‌شود شبکه تولیدشده در اجراهای مختلف یکسان باشد.

تابع create_adjacency_matrix

این تابع گراف تولیدشده را به ماتریس مجاورت تبدیل می‌کند. درایه‌های ماتریس، هزینه یال‌های موجود را نگهداری می‌کنند و برای یال‌های ناموجود مقدار صفر قرار می‌گیرد.

تابع generate_od_true

این تابع ماتریس واقعی مبدا مقصد را تولید می‌کند. هر درایه غیر قطری یک عدد صحیح تصادفی در محدوده مشخص شده در تنظیمات است. سپس با استفاده از np.fill_diagonal تمام درایه‌های قطر اصلی صفر میشوند. برای جلوگیری از وابستگی دنباله تصادفی OD به دنباله تولید شبکه از seed+1 استفاده شده است.

تابع Create_assignment_matrix

این تابع ابتدا فهرست یال‌های شبکه و تمام زوج‌های مبدا مقصد را ایجاد میکند سپس برای هر زوج کوتاه‌ترین مسیر را بر اساس هزینه یال‌ها با تابع networkx.shortest_path محاسبه میکند. پس از استخراج یال‌های هر مسیر درایه مربوط در ماتریس P برابر یک قرارداد میشود خروجی این تابع شامل ماتریس تخصیص فهرست یال‌ها فهرست زوج‌های OD و دیکشنری کوتاه‌ترین مسیرهاست.

تابع generate_link_counts

این تابع ابتدا مقادیر غیرقطری OD-true را مطابق ترتیب ستون‌های ماتریس P به بردار تبدیل میکند سپس شمارش واقعی یال‌ها را از رابطه زیر محاسبه میکند.

$$True_link_counts = P \times OD$$

اگر noise_level بیشتر از صفر باشد نویز نرمال با میانگین صفر به شمارش ها اضافه میشود. انحراف معیار نویز برای هر یال متناسب با مقدار شمارش همان یال در نظر گرفته شده است نتایج نویزی گرد و مقادیر منفی نیز صفر میشوند.

خروجی تابع شامل شمارش مشاهده شمارش واقعی و بردار OD واقعی است.

توابع فایل estimator.py

تابع objective_function

این تابع مقدار تابع هدف را محاسبه میکند ابتدا شمارش تخمینی یالها از ضرب ماتریس تخصیص در بردار OD بدست میاید

$$estimatedlinkcounts = P \times OD$$

سپس مجموع مربعات اختلاف میان شمارش تخمینی و مشاهده شده محاسبه میشود. جمله منظم سازی نیز برابر λ ضرب در مجموع مربعات درایه های بردار OD است خروجی تابع مجموع خطای بازتولید و خطای منظم سازی است.

تابع estimate_od

این تابع مسئله بهینه سازی را حل می کند. بردار اولیه شامل صفر برای تمام زوج های OD است. برای تمام متغیرها کران (0,∞) تعریف می شود تا مقدار منفی تولید نشود.

بهینه سازی با تابع scipy.optimize.minimize و روش L-BFGS-B انجام می شود. اگر بهینه ساز موفق نباشد، تابع یک خطای RuntimeError ایجاد می کند. در صورت موفقیت، بردار OD تخمینی بازگردانده می شود

تابع Vector_to_od_matrix

این تابع بردار OD تخمینی را با استفاده از ترتیب موجود در od-pairs به یک ماتریس مربعی تبدیل میکند درایه های مربوط به هر زوج در محل مناسب قرار میگیرند و قطر اصلی صفر باقی میماند.

تابع evaluate_od

این تابع مقادیر واقعی و تخمینی مربوط به تمام زوج های غیرقطری را استخراج کرده و اختلاف آن ها را محاسبه می کند.

تابع evaluate_link_counts

این تابع شمارش یال های بازتولید شده را از ضرب PPP در بردار OD تخمینی محاسبه می کند. سپس اختلاف آن با شمارش مشاهده شده به دست می آید.

توابع فایل Vrp.py

تابع calculate_demands

این تابع مجموع ستون های ماتریس OD را محاسبه می کند. حاصل این جمع، تقاضای ورودی هر مقصد است. سپس تقاضای گره دپو برابر صفر قرار داده می شود.

تابع create_distance_matrix

این تابع کوتاه ترین فاصله میان تمام زوج گره ها را با الگوریتم Floyd-Warshall محاسبه می کند. خروجی یک ماتریس فاصله است که در حل VRP و محاسبه هزینه مسیرها استفاده می شود.

تابع calculate_route_cost

این تابع هزینه یک مسیر را محاسبه می کند. برای هر دو گره متوالی در مسیر، فاصله متناظر از ماتریس فاصله خوانده شده و به هزینه کل اضافه می شود.

تابع `calculate_total_cost`

این تابع هزینه مسیر تمام خودروها را جمع کرده و هزینه کل ناوگان را برمیگرداند.

تابع `validate_routes`

این تابع اعتبار جواب VRP را بررسی می‌کند.

فایل `main.py` تابع مستقلی تعریف نمی‌کند و وظیفه آن اجرای کامل جریان پروژه است.

آزمایش کد:

نتایج نشان می‌دهد روش پیشنهادی توانسته شمارش یال‌ها را با خطای بسیار کم بازتولید کند و برای تقاضاهای واقعی و تخمینی، مسیرهای معتبر ارائه دهد. همچنین بررسی مسیر تخمینی تحت تقاضای واقعی، اثر خطای تخمین بر محدودیت ظرفیت را به خوبی آشکار کرد؛ بنابراین اهداف هر دو بخش مسئله با موفقیت پیاده‌سازی و ارزیابی شده‌اند.